Advanced Transport Phenomena

Practical session 5 - Multicomponent diffusion

Python edition

Alberto Cuoci and Adrien Guillonneau

2026

The point of this practical is to see why Fick's law, perfectly fine for a two-species mixture, stops being reliable once a third species enters the picture, and to build the two standard fixes: a corrected Fick's law, and the fully rigorous Maxwell-Stefan equations. The case is a real experiment (Duncan and Toor, 1962), with real measured data to compare against. Based on a lecture demonstration by Edoardo Cipriano.

Contents

1 Python notes for this practical **2**

2 Case description **2**

3 Numerical setup **2**

4 Classical Fick's law **3**

5 Corrected Fick's law **4**

6 Maxwell-Stefan equations **5**

7 Generalized Fick's law **6**

8 Bringing it together **7**

9 Others gases: estimation of diffusivity (Chapman-Enskog theory) **7**

10 Exercises **10**

1 Python notes for this practical

This practical is different from the previous ones: there is no spatial grid at all. The system is two well-mixed bulbs connected by a thin capillary, so every quantity only depends on time, a system of ordinary differential equations, not a partial differential equation.

For a system like this, well-behaved and not stiff, the standard tool is a library ODE solver rather than a hand-rolled Euler step. `scipy.integrate.solve_ivp` is the direct equivalent of MATLAB's `ode45`: given a function returning the right-hand side $dx/dt = f(t, x)$, a time span, and an initial condition, it integrates the system automatically, choosing its own step size. Its default method, RK45, evaluates the right-hand side a few times per step and compares two different-order estimates to decide, on its own, how big the next step should be.

```
from scipy.integrate import solve_ivp
sol = solve_ivp(f, [0, t_final], x0, t_eval=t_eval)
```

2 Case description

Two rigid bulbs of equal volume, $V_1 = V_2$, are connected by a thin capillary of length L and cross-section A, following the two-bulb diffusion cell used by Duncan and Toor. Both are filled with a ternary gas mixture of hydrogen (H_2), nitrogen (N_2), and carbon dioxide (CO_2), at the same pressure and temperature, but with different compositions:

$$x_1(t=0) = [0,\ 0.50,\ 0.50] \qquad x_2(t=0) = [0.50,\ 0.50,\ 0]$$

ordered as [H_2, N_2, CO_2]. Notice that **nitrogen starts at the same mole fraction in both bulbs**. Under plain Fick's law, no concentration gradient means no driving force, so nitrogen should not move at all. Experimentally, it does. That single fact is the entire reason this practical exists: a species can diffuse without its own gradient, driven instead by the gradients of the other species, a phenomenon called **osmotic diffusion**.

The real apparatus was small: two bulbs of about 78 cc each, connected by an 8.6 cm capillary just 2 mm wide, with a stopcock, held in a water bath. Each experimental point comes from a separate run: the stopcock is opened at $t = 0$, closed again after a chosen duration, and the gas is sent for chromatographic analysis, which destroys the sample. So the scattered points on the plots below are five independent experiments, not five readings of one continuous run.

The capillary is thin and long enough that diffusion through it can be treated as quasi-steady and one-dimensional, with a linear composition profile between the two bulb compositions. Whatever the diffusive flux J through the capillary turns out to be (a 3-component vector, one entry per species, from bulb 1 to bulb 2), each bulb's composition evolves by a simple mass balance:

$$\frac{dx_1}{dt} = -\frac{J}{C_{tot}}\frac{A}{V_1} \qquad \frac{dx_2}{dt} = +\frac{J}{C_{tot}}\frac{A}{V_2}$$

Everything that follows is really just four different ways of computing J from the current compositions x_1 and x_2.

3 Numerical setup

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

L = 8.59e-2     # capillary length [m]
```

```
d = 2.08e-3     # capillary diameter [m]
V1 = 78.63e-6   # bulb 1 volume [m3]
V2 = 78.63e-6   # bulb 2 volume [m3]
P = 1e5         # pressure [Pa]
T = 35 + 273    # temperature [K]
t_final = 20 * 3600 # final time [s]

# initial mole fractions, ordered [H2, N2, CO2]
x1_init = np.array([0.0 , 0.50, 0.50])
x2_init = np.array([0.50, 0.50, 0.0 ])

# binary diffusion coefficients D[i,j] [m2/s], symmetric, diagonal unused
D = np.array([[0.0,   8.33e-5, 6.80e-5],
              [8.33e-5, 0.0,   1.68e-5],
              [6.80e-5, 1.68e-5, 0.0 ]])

A = np.pi * d**2 / 4      # capillary cross-section [m2]
Ctot = P / 8314.0 / T     # total molar concentration [kmol/m3]

expdata = np.loadtxt("ExpDuncanToor.exp", skiprows=1)
t_exp = expdata[:, 0] * 3600 # experimental times, converted to seconds

x0 = np.concatenate([x1_init, x2_init])
t_eval = np.linspace(0, t_final, 300)
```

4 Classical Fick's law

Fick's law assumes each species diffuses independently, in proportion to its own gradient only, using an effective diffusivity in the mixture. The standard mixing rule (Wilke's formula) weights the binary diffusivities with respect to every other species:

$$\Gamma_{i,mix} = \frac{1 - x_i}{\sum_{k \neq i} x_k / D_{ik}}$$

and the flux of species i through the capillary follows ordinary Fick's law, driven by the mole fraction difference between the two bulbs:

$$J_i = -C_{tot}\, \Gamma_{i,mix}\, \frac{x_{2,i} - x_{1,i}}{L}$$

Nothing forces the three fluxes to add up to zero here, which they must, physically, since the bulbs are closed. That identity only holds automatically if all three diffusivities happen to be equal, which they are not.

```
def fick_diffusion_coefficients(X, D):
    '''Wilke mixture-averaged diffusivity, one value per species.'''
    n = len(X)
    Dmix = np.zeros(n)
    for i in range(n):
        s = 0.0
        for k in range(n):
            if k != i:
                s += X[k] / D[i, k]
        Dmix[i] = (1 - X[i]) / s
    return Dmix
```

```
def fick_rhs(t, x):
    x1 = x[0:3]
    x2 = x[3:6]
    X = 0.5 * (x1 + x2)

    Dmix = fick_diffusion_coefficients(X, D)
    J = -Ctot * Dmix * (x2 - x1) / L

    dx1_dt = -J / Ctot * A / V1
    dx2_dt = J / Ctot * A / V2
    return np.concatenate([dx1_dt, dx2_dt])


sol_fick = solve_ivp(fick_rhs, [0, t_final], x0, t_eval=t_eval)
x1_fick = sol_fick.y[0:3, :].T
x2_fick = sol_fick.y[3:6, :].T
```

```
species_latex = [r"$H_2$", r"$N_2$", r"$CO_2$"]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for k in range(3):
    axes[k].plot(sol_fick.t / 3600, x1_fick[:, k], color="C1", label="bulb 1")
    axes[k].plot(sol_fick.t / 3600, x2_fick[:, k], color="C0", label="bulb 2")
    axes[k].scatter(t_exp / 3600, expdata[:, 1 + 2*k], color="C1", marker="o")
    axes[k].scatter(t_exp / 3600, expdata[:, 2 + 2*k], color="C0", marker="o")
    axes[k].set_xlabel("time [hr]")
    axes[k].set_ylabel("mole fraction")
    axes[k].set_title(f"{species_latex[k]}, classical Fick")
    axes[k].legend()
plt.tight_layout()
plt.show()
```

5 Corrected Fick's law

The fix: measure how far the fluxes are from summing to zero, and spread that excess back across the species, weighted by how abundant each one currently is. This is the "correction velocity" approach: a small extra transport velocity, common to every species, chosen specifically to force the sum of fluxes back to zero.

$$J_c = -\sum_i J_i \qquad\qquad J_i^{corrected} = J_i + \frac{x_{1,i} + x_{2,i}}{2} J_c$$

```
def fick_corrected_rhs(t, x):
    x1 = x[0:3]
    x2 = x[3:6]
    X = 0.5 * (x1 + x2)

    Dmix = fick_diffusion_coefficients(X, D)
    J = -Ctot * Dmix * (x2 - x1) / L

    Jc = -np.sum(J)
    J = J + 0.5 * (x1 + x2) * Jc

    dx1_dt = -J / Ctot * A / V1
    dx2_dt = J / Ctot * A / V2
```

```
    return np.concatenate([dx1_dt, dx2_dt])


sol_corr = solve_ivp(fick_corrected_rhs, [0, t_final], x0, t_eval=t_eval)
x1_corr = sol_corr.y[0:3, :].T
x2_corr = sol_corr.y[3:6, :].T
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for k in range(3):
    axes[k].plot(sol_fick.t / 3600, x1_fick[:, k], "--", color="C1", label="Fick, bulb
        1")
    axes[k].plot(sol_fick.t / 3600, x2_fick[:, k], "--", color="C0", label="Fick, bulb
        2")
    axes[k].plot(sol_corr.t / 3600, x1_corr[:, k], color="C1", label="corrected Fick,
       bulb 1")
    axes[k].plot(sol_corr.t / 3600, x2_corr[:, k], color="C0", label="corrected Fick,
       bulb 2")
    axes[k].scatter(t_exp / 3600, expdata[:, 1 + 2*k], color="C1", marker="o")
    axes[k].scatter(t_exp / 3600, expdata[:, 2 + 2*k], color="C0", marker="o")
    axes[k].set_xlabel("time [hr]")
    axes[k].set_ylabel("mole fraction")
    axes[k].set_title(f"{species_latex[k]}, Fick vs corrected Fick")
axes[0].legend(fontsize=8)
plt.tight_layout()
plt.show()
```

6 Maxwell-Stefan equations

Both models above still treat each species as if it only cared about its own gradient. The Maxwell-Stefan equations instead couple every species' flux to every gradient at once, which is the only way to correctly capture a species diffusing with no gradient of its own. For this problem, written per bulb pair, the equations reduce to one linear system for the three fluxes, $M\,J = b$.

Two of the three rows come directly from the Maxwell-Stefan relations (one per non-reference species), and the third encodes flux closure, $J_{H_2} + J_{N_2} + J_{CO_2} = 0$, replacing the one equation that is not independent of the other two. The matrix coefficients for rows 0 and 1 (species H_2 and N_2) are, for column j:

$$M_{ij} = -\frac{x_i}{C_{tot}\,D_{ij}} \quad (j \neq i) \qquad\qquad M_{ii} = \sum_{k \neq i} \frac{x_k}{C_{tot}\,D_{ik}}$$

with right-hand side $b_i = -\dfrac{x_{2,i} - x_{1,i}}{L}$ for $i = 0, 1$, and the closure row $M_{2,:} = [1, 1, 1]$, $b_2 = 0$.

```
def maxwell_stefan_rhs(t, x):
    x1 = x[0:3]
    x2 = x[3:6]
    X = 0.5 * (x1 + x2)
    dx_dz = (x2 - x1) / L

    n = 3
    M = np.zeros((n, n))
    b = np.zeros(n)

    M[-1, :] = 1.0
    b[-1] = 0.0
```

```
    for i in range(n - 1):
        for j in range(n):
            if i != j:
                M[i, j] = -X[i] / (Ctot * D[i, j])
            else:
                s = 0.0
                for k in range(n):
                    if k != i:
                        s += X[k] / (Ctot * D[i, k])
                M[i, j] = s
        b[i] = -dx_dz[i]

    J = np.linalg.solve(M, b)

    dx1_dt = -J / Ctot * A / V1
    dx2_dt = J / Ctot * A / V2
    return np.concatenate([dx1_dt, dx2_dt])


sol_ms = solve_ivp(maxwell_stefan_rhs, [0, t_final], x0, t_eval=t_eval)
x1_ms = sol_ms.y[0:3, :].T
x2_ms = sol_ms.y[3:6, :].T
```

7 Generalized Fick's law

The Maxwell-Stefan equations can be rewritten in a form that looks like Fick's law again, $J = -C_{tot}\,\Gamma\,\nabla x$, but with a full matrix Γ instead of one number per species, which is what actually makes it capable of reproducing osmotic and reverse diffusion. The matrix comes from inverting a closely related system built the same way as the Maxwell-Stefan matrix above, but only for the two independent gradients (the third species again serving as the reference).

```
def fick_generalized_rhs(t, x):
    x1 = x[0:3]
    x2 = x[3:6]
    X = 0.5 * (x1 + x2)
    dx_dz = (x2 - x1) / L

    n = 3
    B = np.zeros((n - 1, n - 1))
    for i in range(n - 1):
        for j in range(n - 1):
            if i != j:
                B[i, j] = -X[i] * (1/D[i, j] - 1/D[i, n-1])
            else:
                s = X[i] / D[i, n-1]
                for k in range(n):
                    if k != i:
                        s += X[k] / D[i, k]
                B[i, j] = s

    Gamma = np.eye(n - 1)
    Dmix = np.linalg.solve(B, Gamma)

    J = np.zeros(n)
    J[0] = -Ctot * (Dmix[0, 0]*dx_dz[0] + Dmix[0, 1]*dx_dz[1])
    J[1] = -Ctot * (Dmix[1, 0]*dx_dz[0] + Dmix[1, 1]*dx_dz[1])
```

```
    J[2] = -J[0] - J[1]

    dx1_dt = -J / Ctot * A / V1
    dx2_dt = J / Ctot * A / V2
    return np.concatenate([dx1_dt, dx2_dt])


sol_gen = solve_ivp(fick_generalized_rhs, [0, t_final], x0, t_eval=t_eval)
x1_gen = sol_gen.y[0:3, :].T
x2_gen = sol_gen.y[3:6, :].T
```

8 Bringing it together

All four models, against the real measurements, one panel per species. Look specifically at the nitrogen panel: classical Fick predicts essentially no early movement, while Maxwell-Stefan and generalized Fick show nitrogen moving right from the start, matching the data. That is osmotic diffusion, made visible.

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
models = [("Fick", sol_fick.t, x1_fick, x2_fick, "--"),
          ("Corrected Fick", sol_corr.t, x1_corr, x2_corr, ":"),
          ("Maxwell-Stefan", sol_ms.t, x1_ms, x2_ms, "-"),
          ("Generalized Fick", sol_gen.t, x1_gen, x2_gen, "-.")]

for k in range(3):
    for name, t, xx1, xx2, style in models:
        axes[k].plot(t/3600, xx1[:, k], style, color="C1", label=f"{name}, bulb 1")
        axes[k].plot(t/3600, xx2[:, k], style, color="C0", label=f"{name}, bulb 2")
    axes[k].scatter(t_exp/3600, expdata[:, 1+2*k], color="k", zorder=5, label="
        experiment")
    axes[k].scatter(t_exp/3600, expdata[:, 2+2*k], color="k", zorder=5)
    axes[k].set_xlabel("time [hr]")
    axes[k].set_ylabel("mole fraction")
    axes[k].set_title(species_latex[k])

axes[0].legend(fontsize=7, loc="best")
plt.tight_layout()
plt.show()
```

9 Others gases: estimation of diffusivity (Chapman-Enskog theory)

So far the binary diffusivities `D` were given: they were measured by Duncan and Toor. When no measurement exists, the usual engineering path is to find an established theory for the regime at hand, gather the basic tabulated data it needs, apply it, and validate it against a case already trusted before using it anywhere new. Here that theory is kinetic theory for a dilute gas (Chapman-Enskog), valid since the system is at a few bar and non-polar:

$$D_{AB} = \frac{0.00266\, T^{3/2}}{P\, M_{AB}^{1/2}\, \sigma_{AB}^{2}\, \Omega_D(T^*)} \qquad \text{with} \qquad M_{AB} = \frac{2}{1/M_A + 1/M_B}$$

with D_{AB} in cm^2/s, T in K, P in bar, σ in Å and M in g/mol. The molecules are modeled with a Lennard-Jones potential, size σ and well depth ε/k, fitted beforehand to other measured properties. For a pair of unlike molecules, the usual combining rules are

$$\sigma_{AB} = \frac{\sigma_A + \sigma_B}{2}, \qquad \varepsilon_{AB} = \sqrt{\varepsilon_A\,\varepsilon_B}, \qquad T^* = \frac{kT}{\varepsilon_{AB}}$$

and the collision integral $\Omega_D(T^*)$, equal to 1 for hard spheres, corrects for the real attraction between molecules (the Neufeld fit):

$$\Omega_D = \frac{1.06036}{T^{*\,0.1561}} + \frac{0.19300}{e^{0.47635\,T^*}} + \frac{1.03587}{e^{1.52996\,T^*}} + \frac{1.76474}{e^{3.89411\,T^*}}$$

Valid for dilute, non-polar gases, typically within 5-10% accuracy.

```
LJ = {
    "H2": (2.968, 33.3),
    "N2": (3.681, 91.5),
    "O2": (3.433, 113.0),
    "CO2": (3.996, 190.0),
    "CH4": (3.822, 137.0),
}
MW = {"H2": 2.016, "N2": 28.014, "O2": 31.998, "CO2": 44.010, "CH4": 16.043}

def omega_D(Tstar):
    """Collision integral for diffusion (Neufeld fit)."""
    return (1.06036 / Tstar**0.15610
            + 0.19300 / np.exp(0.47635 * Tstar)
            + 1.03587 / np.exp(1.52996 * Tstar)
            + 1.76474 / np.exp(3.89411 * Tstar))

def chapman_enskog_D(A, B, T, P):
    """Binary diffusivity of the pair A-B [m2/s]. T in K, P in Pa."""
    sigma_A, eps_A = LJ[A]
    sigma_B, eps_B = LJ[B]
    sigma_AB = 0.5 * (sigma_A + sigma_B)
    eps_AB = np.sqrt(eps_A * eps_B)
    M_AB = 2.0 / (1.0 / MW[A] + 1.0 / MW[B])
    Tstar = T / eps_AB
    D_AB = 0.00266 * T**1.5 / ((P / 1e5) * np.sqrt(M_AB) * sigma_AB**2 * omega_D(Tstar
        ))
    return D_AB * 1e-4 # cm2/s to m2/s

def D_matrix(species, T, P):
    """Symmetric matrix of binary diffusivities [m2/s] (diagonal unused)."""
    n = len(species)
    Dm = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
                Dm[i, j] = chapman_enskog_D(species[i], species[j], T, P)
    return Dm
```

The estimate is checked against the real H_2-N_2-CO_2 diffusivities before trusting it on anything new:

```
D_H2_measured = D.copy()
D_ce_H2 = D_matrix(["H2", "N2", "CO2"], T, P)

print(f"{'pair':8s} {'measured':>10s} {'Chapman-Enskog':>16s} {'error':>8s} [cm2/s]")
for i, j, name in [(0, 1, "H2-N2"), (0, 2, "H2-CO2"), (1, 2, "N2-CO2")]:
    meas = D_H2_measured[i, j] * 1e4
    est = D_ce_H2[i, j] * 1e4
    print(f"{name:8s} {meas:10.3f} {est:16.3f} {100 * (est / meas - 1):7.1f}%")
```

Replacing hydrogen by oxygen or methane, both much closer to nitrogen and carbon dioxide in size and mass, gives a mixture where the three binary diffusivities are much closer to each other. Before running anything, it is worth predicting from that alone whether the ternary effects should grow or shrink.

```
D_O2 = D_matrix(["O2", "N2", "CO2"], T, P)
D_CH4 = D_matrix(["CH4", "N2", "CO2"], T, P)

cases = {"H2-N2-CO2 (measured)": D_H2_measured,
         "O2-N2-CO2": D_O2,
         "CH4-N2-CO2": D_CH4}

print(f"{'mixture':22s} {'D12':>7s} {'D13':>7s} {'D23':>7s} {'max/min':>9s} [cm2/s]")
for name, Dm in cases.items():
    d12, d13, d23 = Dm[0, 1] * 1e4, Dm[0, 2] * 1e4, Dm[1, 2] * 1e4
    print(f"{name:22s} {d12:7.3f} {d13:7.3f} {d23:7.3f} {max(d12, d13, d23) / min(d12,
        d13, d23):9.1f}")
```

```
def run_models(D_new, t_end):
    """Run Fick, corrected Fick and Maxwell-Stefan with another diffusivity matrix."""
    global D
    D_saved = D
    D = D_new
    try:
        t_eval_new = np.linspace(0, t_end, 300)
        out = {}
        for name, f in [("Fick", fick_rhs),
                        ("Corrected Fick", fick_corrected_rhs),
                        ("Maxwell-Stefan", maxwell_stefan_rhs)]:
            out[name] = solve_ivp(f, [0, t_end], x0, t_eval=t_eval_new, rtol=1e-8, atol
                =1e-10)
    finally:
        D = D_saved
    return out

t_final_slow = 100 * 3600  # slower gases: longer experiment [s]
results = {}
for name, Dm in cases.items():
    results[name] = run_models(Dm, t_final if "H2" in name else t_final_slow)
```

The ratio between the largest and smallest D_{ij} is the single number that predicts how strong the ternary coupling will be, about 5 for hydrogen, about 1.4 for oxygen or methane. Fick's law is not wrong in general, it is the limit of Maxwell-Stefan when the diffusivities happen to be similar, and it is exactly right in a binary mixture.

10 Exercises

1. Plot the three fluxes J_{H_2}, J_{N_2}, J_{CO_2} over time for the classical Fick model, and confirm they do not sum to zero. Repeat for the corrected model and confirm they do.

2. At $t = 0$, print the nitrogen flux predicted by classical Fick and by Maxwell-Stefan. Explain, in one or two sentences, why they differ so much given that nitrogen's own gradient is exactly zero in both models.

3. The reference species in both the Maxwell-Stefan and generalized Fick assembly is CO_2 (species index 2). Rebuild both using H_2 (index 0) as the reference instead, and confirm the predicted mole fractions do not change.

4. Change the initial conditions so that nitrogen also starts with a gradient, for example $x_1 = [0, 0.3, 0.7]$, $x_2 = [0.5, 0.5, 0]$, and compare how differently the four models respond now that every species has a gradient to work with.